

LLM Notes & Wiki

Karpathy's Zero to Hero, nanoGPT & LLM Concepts

Andrej Karpathy | karpathy.ai | github.com/karpathy

Introduction

Andrej Karpathy, former Director of AI at Tesla and founding member/research scientist at OpenAI, has produced some of the most influential educational resources on large language models. This document summarizes the key concepts from his 'Zero to Hero' lecture series, blog posts, and the LLM wiki/notes he has shared publicly.

1. The Neural Network Series: Zero to Hero

Karpathy's YouTube series (karpathy.ai/zero-to-hero.html) builds intuition from the ground up:

micrograd: Builds a scalar-valued autograd engine from scratch in ~150 lines of Python. Demonstrates backpropagation through a DAG. Foundation for understanding gradient descent.

makemore: Character-level language models from scratch. Progresses through bigrams → MLP → RNN → GRU → Transformer. Introduces batch normalization, embedding tables, context windows.

nanoGPT: Complete GPT-2 (124M) reproduction. Covers tokenization (BPE), attention blocks, positional encoding, layer normalization, weight tying, training loops, and sampling.

minbpe: Minimal BPE tokenizer from scratch. Covers the byte-pair encoding algorithm used in GPT-2/GPT-4 tokenizers.

Intro to LLMs (1hr): High-level overview: pretraining, supervised finetuning, RLHF, emerging capabilities, scaling laws, jailbreaks, and future directions.

2. Key Concepts from Karpathy's LLM Notes

2.1 Tokenization

Tokenizers split text into subword tokens. GPT-4 uses ~100k BPE tokens. Issues: numbers split oddly, spaces before words matter, non-English languages use more tokens per word, code indentation can fragment poorly. Karpathy demonstrates that tokenization bugs are a major source of model quirks.

2.2 The Residual Stream

The Transformer can be viewed as manipulating a 'residual stream' of vectors. Each attention head and MLP layer reads from and writes to this stream additively. This view (from Elhage et al., 'A Mathematical Framework for Transformer Circuits') enables mechanistic interpretability.

2.3 Scaling Laws (Chinchilla)

Hoffmann et al. (2022) showed that for a compute budget C , the optimal model has $\sim C^{0.5}$ parameters and is trained on $\sim C^{0.5}$ tokens. This 'Chinchilla scaling' implies most prior models were undertrained for their size. Llama 2 (7B) was trained on 2T tokens, following Chinchilla-optimal principles.

$$N_{\text{opt}} \approx 0.2 * C^{0.5} \text{ (parameters)}$$

$$D_{\text{opt}} \approx 5.0 * C^{0.5} \text{ (tokens)}$$

2.4 RLHF Pipeline

The three-stage process described in Karpathy's talks:

- Stage 1 — Pretraining: Next-token prediction on internet text (~10T tokens for GPT-4 class models)
- Stage 2 — SFT (Supervised Fine-tuning): Human-written (prompt, response) pairs teach instruction following
- Stage 3 — RLHF: Reward model trained on human preferences; PPO policy optimization to maximize reward
- Stage 3b — DPO (Direct Preference Optimization): More stable alternative to PPO; directly optimizes policy

3. nanoGPT Architecture (Summary)

```
# Key nanoGPT hyperparameters for GPT-2 (124M) reproduction
n_layer      = 12      # transformer blocks
n_head       = 12      # attention heads per block
n_embd       = 768     # embedding dimension
block_size   = 1024    # context window (tokens)
vocab_size    = 50257   # BPE vocabulary (GPT-2 tokenizer)
dropout       = 0.0     # no dropout for inference

# Training config (Karpathy's reproduction)
batch_size    = 0.5M tokens # tokens per step
learning_rate = 6e-4        # peak LR with cosine decay
weight_decay  = 0.1
beta1, beta2  = 0.9, 0.95   # AdamW betas
grad_clip     = 1.0
total_tokens  = 10B          # ~1hr on 8x A100
```

4. LLM.c: Pure C/CUDA Training

In 2024, Karpathy released llm.c — GPT-2 training in ~1000 lines of C/CUDA. It achieves parity with PyTorch at ~7% faster throughput and demonstrates that modern deep learning frameworks, while convenient, have significant overhead. The repo grew into a community project reproducing GPT-2 training in under 3 minutes on 8x H100s (from 45 minutes originally) — the 'NanoGPT Speedrun'.

5. Key Advice from Karpathy

- Always visualize your data before training. Most bugs are in preprocessing, not the model.
- Start with the simplest possible baseline and add complexity only when needed.
- Use AdamW, not SGD, for transformer training — momentum matters.
- Layer normalization before (Pre-LN) rather than after (Post-LN) attention is more stable.
- Flash Attention is a free 2–4x speedup; use it by default.
- LLMs are 'stochastic parrots' in some ways but also generalize — both are true.